

ML Workflow Helper

A step by step web app for machine learning on any CSV file

Project guide: idea rating, design, how each step works, how to run it, and what to improve next

1. Idea rating

Overall: 8 out of 10 as a personal tool and as a portfolio project.

Area	Score	Reason
Usefulness	9/10	You will really use it. Every new dataset needs the same steps, and a click-through interface saves a lot of repeated notebook work.
Learning value	9/10	It forces you to understand the full pipeline: cleaning, splitting, encoding, scaling, training, tuning, evaluation and saving.
Portfolio value	8/10	A working tool that handles both classification and regression looks much stronger than one more single-dataset notebook.
Originality	6/10	Tools like PyCaret, AutoViz and various Streamlit AutoML apps already exist. The value is in building your own and understanding it.
Difficulty and scope	7/10	Big but realistic. The main risk is trying to support every edge case at once.

What already makes the idea good

- A clear order of steps, with a choice at every step, so the user stays in control.
- Both classification and regression, and both default and tuned training.
- Real evaluation, not just accuracy: confusion matrix, sensitivity, specificity, ROC AUC and error ratios.
- A save option that keeps the preprocessing together with the model.

What would move it from 8 to 9 or 10

- Fit imputation, encoding and scaling inside a scikit-learn Pipeline so nothing from the test data leaks into training (the app already fits the encoder and scaler on train only; imputation is the remaining gap).
- Compare models with cross validation, not a single split, so rankings are more trustworthy.
- Add feature importance or SHAP, and a page where a user types values and gets a prediction.
- Add tests and a short README so the project looks professional on GitHub.

Two honest cautions

- Accuracy alone is misleading on imbalanced data. That is why the app also reports F1, sensitivity, specificity and ROC AUC, and offers stratified splitting.
- Filling missing values before the split leaks a small amount of information. This is fine for a personal tool, but it should be documented (the code has a comment about it).

2. What the app does

The app is one Python file (app.py) built with Streamlit. The user moves through nine steps in the sidebar. Nothing is changed until the user clicks an apply button, and every choice is written to a log.

Step	What the user sees	Choices the user makes
1. Upload and target	Data preview, guessed problem type	Separator, target column, classification or regression
2. Overview	head, tail, columns, shape, dtypes, info, describe	Number of rows, include text columns, drop useless columns
3. Missing values	Table of missing counts and percentages	Only check, or handle: column drop threshold, numeric and text strategy
4. Duplicates	Duplicate count and sample rows	Only check, drop keep first, drop keep last
5. Visualization	Target plot, histograms, boxplots, correlation heatmap, countplots	Which plots, how many columns
6. Split, encode, scale	Shapes and previews of X_train, X_test, y_train	Test size, random state, stratify, One-Hot or Ordinal, Standard / MinMax / Robust / None
7. Train models	Metrics, report, plots per model	Default or tuned, which algorithms, search tries, CV folds
8. Compare	Ranked table and bar chart, best model, overfitting warning	Metric used for ranking
9. Save	Download button	Pickle of best model, or zip of the entire work without the model

3. Models and tuning

Classification	Regression
Logistic Regression	Linear Regression
Random Forest	Random Forest
Decision Tree	Decision Tree
SVM	SVR
CatBoost	CatBoost
AdaBoost	AdaBoost
LightGBM	LightGBM

Default mode trains each chosen model with library defaults (a few fixed settings such as max_iter or probability output for SVM). It is fast and is a good baseline.

Tuned mode uses RandomizedSearchCV with a small search space per model and cross validation. Random search was chosen over full grid search because it finds good settings in far less time, which matters for SVM and CatBoost. Classification tuning scores by accuracy and regression by R2. The best parameters and cross validation score are shown next to the results.

Results from default and tuned runs are stored under separate names, so you can compare a default Random Forest with a tuned Random Forest in the same table.

4. Metrics explained

Classification

Metric	Meaning
Train and test accuracy	Share of correct predictions. A big gap means the model memorised the training data (overfitting).
Classification report	Precision, recall and F1 for each class.
Confusion matrix	Counts of correct and wrong predictions for each class pair.
Sensitivity	True positives divided by all actual positives (same as recall). For more than two classes it is the average of one-vs-rest values.
Specificity	True negatives divided by all actual negatives. Same averaging rule for multiclass.
ROC AUC and curve	How well the model ranks positives above negatives across all thresholds. 0.5 is random guessing, 1.0 is perfect.

Regression

Metric	Meaning
R2	Share of variance explained. Closer to 1 is better; negative means worse than predicting the mean.

Metric	Meaning
RMSE	Typical error size in the same unit as the target. Punishes large errors more.
MAE	Average absolute error.
RSE (relative squared error)	Sum of squared errors divided by the squared error of always predicting the mean. Below 1 beats the mean; it equals 1 minus R2.
RAE (relative absolute error)	Same idea using absolute errors. Below 1 beats the mean.

Note: RSE is sometimes used to mean residual standard error. In this project it always means relative squared error.

5. How to run it

Keep the files flat in one folder: app.py and requirements.txt.

```
pip install -r requirements.txt
streamlit run app.py
```

The app opens in the browser at <http://localhost:8501>. Upload a CSV, pick the target and follow the sidebar from step 1 to step 9.

Using the saved pickle later

The pickle is a bundle that holds the model, encoder, scaler, label encoder, column names and class names. The app shows a ready snippet in Step 9 that loads the bundle and prepares new data the same way as training. Only load pickle files you created yourself, and use the same library versions, because pickles are not safe or stable across different environments.

6. Design decisions worth knowing

- Encoder and scaler are fitted on the training part only and then applied to the test part, so test data never influences preprocessing.
- Rows with a missing target are always dropped, never filled.
- The problem type is guessed (text or 10 or fewer unique values means classification) but the user can override it.
- Changing an earlier step clears the split and the trained models, so results never belong to stale data.
- Scaling is applied to all columns after encoding, following the workflow you described. Tree models do not need scaling, but it does no harm.

7. Known limits and next improvements

- Missing value filling happens before the split (small leak). Move it into the split step or a Pipeline.
- Very large files and heavy tuning can be slow. Add a row sample option and a time limit.
- One-Hot encoding on columns with hundreds of categories creates too many columns. Add a warning or target encoding.
- Only CSV input for now. Add Excel and sample datasets.
- Add a prediction page, feature importance, and cross validated comparison.
- Add unit tests for the metric functions, especially specificity and the multiclass ROC AUC.

8. Suggested build order if you extend it

- Get the whole flow working on two small datasets, one classification and one regression.
- Add the prediction page and feature importance.
- Refactor into a scikit-learn Pipeline.
- Write the README with screenshots and push to GitHub.